Bayesian Belief Network-Driven Proactive Fault Tolerance for Microservice Placement in Fog Computing Environments

Rahul Akshath Rajendran
3122235002095
rahul2310405@ssn.edu.in
Mohit R
3122235002072
mohit2310893@ssn.edu.in
Dr. Sabireen H, Assistant Professor
Project Guide
sabireenh@ssn.edu.in
Department of Information Technology
SSN College of Engineering, Kalavakkam, Chennai 603110, India

Abstract

Fog computing extends cloud services to the network edge in order to satisfy the stringent latency requirements of Internet-of-Things (IoT) applications such as remote cardiac monitoring, intelligent video surveillance, and immersive gaming. The fog nodes that host these applications are, however, heterogeneous, resource-constrained, and considerably less reliable than data-centre hardware, so the failure of a single node can breach the deadline of a safety-critical service. Existing fog fault-tolerance mechanisms are predominantly reactive: checkpoint-and-restore, primary-backup replication, acknowledgement-based retransmission, and heartbeat-driven failover all take effect only after a fault has already manifested, and therefore incur a recovery latency that latency-critical workloads cannot absorb. The few proactive schemes reported in the literature trigger migration on a single fixed threshold applied to a single metric, which neither captures the multi-causal nature of fog node degradation nor tolerates missing or noisy telemetry. This paper proposes a Bayesian Belief Network (BBN) driven proactive fault-tolerance framework for microservice placement in fog environments. The framework contributes: (i) a causal BBN that infers the posterior probability of node failure within a look-ahead horizon from partial and noisy telemetry and exposes an explainable causal chain rather than an opaque score; (ii) a risk-aware placement and expected-loss migration policy that replaces fixed thresholds with a cost-sensitive decision rule; and (iii) a criticality-aware selective replication scheme that allocates standby capacity in proportion to predicted risk and application criticality. The framework is being realised as an extension to the iFogSim2 simulator and will be evaluated on the Melbourne CBD edge-resource and user-mobility datasets against representative reactive and threshold-based baselines.

Index Terms

Bayesian belief network, fog computing, proactive fault tolerance, microservice placement, reliability, iFogSim2, Internet of Things, probabilistic inference.

I. Introduction

THE proliferation of latency-sensitive Internet-of-Things (IoT) applications has exposed a structural limitation of the centralised cloud model. An electrocardiogram stream that must be classified within a few tens of milliseconds, or a surveillance feed that must trigger a pan-tilt-zoom actuator before a tracked object leaves the frame, cannot tolerate the round-trip delay to a distant data centre. Fog computing addresses this by interposing a tier of computational resources—gateways, routers, base stations, and micro data centres—between the end devices and the cloud, so that application modules execute within one or two network hops of the sensors that feed them [1], [2].

This proximity is purchased at the cost of reliability. Whereas a cloud data centre offers redundant power, cooling, and network paths, a fog node is frequently a commodity device deployed in an uncontrolled environment, sharing a constrained uplink, operating on limited or battery power, and serving mobile users whose handovers continually reshape the load it carries. Empirical studies of distributed infrastructure consistently report that such nodes fail far more often than data-centre hosts, and that failures are rarely instantaneous: a node typically degrades through a period of rising utilisation, thermal stress, queue build-up, and link deterioration before it becomes unresponsive [3]. That degradation window is precisely the opportunity that a proactive mechanism should exploit.

The fog fault-tolerance literature has, however, largely declined to exploit it. The dominant mechanisms—dynamic checkpointing [4], acknowledgement-based retransmission [5], and heartbeat monitoring with failover to a replicated backup [6], [8]—are reactive by construction. They detect the absence of a response or the loss of a message, and then initiate recovery. For a best-effort workload this is adequate; for a cardiac arrhythmia classifier it means that the very interval during which an alarm should have been raised is consumed by state restoration. Even the schemes that describe themselves as proactive reduce the decision to a single scalar comparison, such as creating a checkpoint whenever CPU utilisation exceeds twenty per cent [4], or migrating a microservice when a composite health index falls below a preset fault threshold [8]. A fixed threshold on a single metric is simultaneously too eager and too late: it fires on transient load spikes that resolve without incident, and it stays silent when a node is degrading along a dimension the metric does not observe.

What is missing is a principled representation of uncertainty. Fog node health is a latent condition, inferred indirectly from several noisy and mutually dependent indicators, some of which may be unavailable at any given instant because a telemetry agent has itself been affected. Bayesian Belief Networks (BBNs) are the canonical formalism for exactly this situation. A BBN encodes causal structure as a directed acyclic graph and quantifies it with conditional probability tables, supports exact inference of a posterior over an unobserved variable given whatever evidence happens to be available, degrades gracefully when evidence is partial by marginalising over the missing variables, and permits expert knowledge to be combined with observed data [9]. BBNs have been applied with documented success to software defect and reliability prediction [10], to dependability, risk analysis, and maintenance in industrial systems [11], and to environmental modelling under sparse observation [12]. Their transfer to fog reliability management is natural and, to the best of our knowledge, unexplored.

This paper proposes a proactive fault-tolerance framework in which a BBN estimates, for every fog node, the probability that it will fail within a look-ahead horizon, and in which that posterior drives placement, migration, and redundancy decisions. The contributions are threefold:

- A BBN-based failure-prediction engine that infers the posterior probability of imminent node failure from heterogeneous telemetry, tolerates missing and noisy evidence, exposes the causal chain responsible for each prediction, and adapts its conditional probability tables online from observed outcomes.
- A risk-aware placement and expected-loss migration policy in which the BBN posterior enters a multi-objective utility over latency, energy, and risk, and in which migration is triggered by a cost-sensitive expected-loss comparison rather than a fixed threshold.
- A criticality-aware selective replication scheme that allocates standby capacity in proportion to the product of predicted risk and application criticality, so that redundancy is spent where it changes the outcome instead of uniformly across all modules.

The remainder of the paper is organised as follows. Section II surveys reactive and proactive fault tolerance in fog computing and the use of BBNs in reliability engineering, and identifies the research gap. Section III presents the proposed framework, its block diagram, and its workflow. Section IV describes the simulation environment, the datasets, and the evaluation plan. Section V concludes and outlines the work planned for Phase II.

II. Literature Survey

A. Reactive Fault Tolerance in Fog Environments

Rajab and Younis [4] propose a dynamic fault-tolerance-aware scheduling approach for healthcare systems on fog computing. Their scheme combines Dynamic Weighted Round Robin (DWRR) scheduling, in which each fog server is assigned a weight derived from its current CPU and memory utilisation, with primary-backup replication and dynamic checkpointing. Rather than checkpointing at fixed intervals, the system creates a checkpoint whenever CPU utilisation crosses a threshold of 0.2, and every checkpoint is replicated to a designated backup server so that execution can resume from the most recent saved state if the primary fails. The work demonstrates a clear throughput benefit from load-aware distribution and a reduced re-execution cost from adaptive checkpointing. Its limitations are equally clear: the threshold is fixed and workload-independent, the backup server consumes storage whether or not a failure ever occurs, evaluation is confined to a healthcare scenario, and the system possesses no capacity to anticipate a failure before the threshold condition is met.

Syed et al. [5] address reliability in vehicular networks through QAFT-SDVN, which integrates software-defined networking with a cloud-fog substrate. A centralised SDN controller classifies each message by type, deadline, and size, assigns a priority accordingly, and routes safety-critical traffic such as ambulance alerts to the nearest fog node while queueing non-safety traffic at lower priority. Fault tolerance is provided by acknowledgement-based retransmission: the sender awaits an ACK and retransmits if none arrives within the expected interval. The approach roughly halves response time for prioritised traffic and reduces the message failure ratio, but recovery is purely reactive and consists solely of retransmission. No backup execution, checkpointing, or replica management is provided, the additional ACK traffic inflates network load, and the design is specialised to the vehicular domain.

Naim, Zain, and Jalil [6] place a fog middleware between users and application servers to improve availability. The middleware polls the primary server with heartbeat requests, maintains a replicated backup containing the same application and data, and on detecting the absence of a heartbeat response activates the backup and transparently redirects all subsequent user requests to it. A prototype recovered from three of four simulated failure scenarios without manual intervention. The mechanism is nevertheless strictly detection-driven: it waits until the server has already stopped responding, considers only a single backup so that a correlated double failure is unrecoverable, imposes continuous heartbeat overhead, and performs no load-aware server selection.

B. Optimisation-Based and Threshold-Based Proactive Placement

Reyana et al. [7] propose QoS-EO, a hybrid task-placement approach that combines Particle Swarm Optimisation with the Artificial Bee Colony algorithm. Candidate solutions encode task-to-node assignments and are scored by a fitness function over response time, energy consumption, reliability, throughput, and execution cost; PSO drives the swarm towards better assignments while ABC's worker, onlooker, and scout phases refine and diversify the search. Better placement reduces node overload and thereby improves fault tolerance indirectly. The cost is computational: running two metaheuristics in sequence is expensive, performance is sensitive to inertia-weight and learning-factor settings, and the search space grows unfavourably as the number of IoT devices increases. Crucially, reliability enters only as a static term in a fitness function; the scheme does not estimate the probability that any particular node will fail.

Gupta and Singh [8] come closest to a proactive design. Their Fault-Tolerance Aware Placement Algorithm (FTAPA) selects a primary fog node using a Dynamic Connections Load Balancer that evaluates active connections, resource availability, and node health; it simultaneously pre-selects a backup node, monitors the health of the primary against a computed fault threshold, and migrates the microservice to the backup before the primary fails outright. This anticipatory migration measurably reduces service downtime relative to detection-driven recovery. The authors themselves identify the residual weaknesses: backup nodes consume memory and processing capacity even when failures never materialise, continuous health monitoring adds overhead, and the fault threshold is difficult to set—too low and migration happens too late, too high and the system migrates unnecessarily. The threshold is a point estimate of a quantity that is inherently uncertain, and no confidence is attached to it.

C. Bayesian Belief Networks for Reliability Prediction

A Bayesian Belief Network is a directed acyclic graph whose vertices are random variables and whose edges encode direct probabilistic dependence, quantified by a conditional probability table for each vertex given its parents. The joint distribution factorises as the product of these local conditionals, which makes exact posterior inference tractable for the sparse structures that arise when causal knowledge is available. The foundational treatment by Pearl [9] established both the semantics of such networks and the message-passing algorithms that make inference practical.

The formalism has been applied extensively to reliability. Fenton, Neil, and Marquez [10] use BNs to predict software defects and reliability, arguing that the ability to combine causal process factors with quantitative measures, and qualitative expert judgement with observed data, overcomes well-known limitations of classical metrics-based models; the approach has been deployed by Motorola, Siemens, and Philips with reported accuracy. Weber et al. [11] survey a decade of BN applications to dependability, risk analysis, and maintenance, and document a sustained growth in adoption attributable to advantages over Markov chains, fault trees, and Petri nets—specifically the capacity to model complex systems with heterogeneous variables, to perform both predictive and diagnostic reasoning, and to update beliefs incrementally as evidence arrives. Aguilera et al. [12] review BNs in environmental modelling, a domain characterised, like fog telemetry, by sparse and unreliable observation, and emphasise the graceful handling of missing data. More recently, Pang and Abramov [13] analyse reliability and parameter selection for IoT communication using deep learning, confirming the importance of predictive reliability modelling for IoT while also illustrating the interpretability cost of black-box predictors—a cost that a BBN does not impose.

Two properties make BBNs particularly appropriate for fog node health. First, inference is well-defined under partial evidence: unobserved variables are marginalised out, so a prediction is still produced when a telemetry channel is unavailable, which is precisely the condition likely to coincide with impending failure. Second, the network is diagnostic as well as predictive: having inferred a high failure probability, the same model identifies which parent variables are most responsible, yielding an explanation that an operator can act on and that a threshold comparison cannot provide.

D. Comparative Analysis

Table I positions the five surveyed mechanisms against the framework proposed in this paper.

TABLE I: Comparative Analysis of Fault-Tolerance Mechanisms in Fog Computing

Reference	Core Mechanism	Paradigm	Limitations Identified
Rajab & Younis (2021) [4]	DWRR scheduling, primary-backup replication, CPU-threshold checkpointing	Reactive	Fixed workload-independent threshold; standby storage consumed whether or not a failure occurs; cannot anticipate failure.
Syed et al. (2022) [5]	SDN priority routing, ACK-based retransmission	Reactive	Recovery is retransmission only; no replication or checkpointing; extra ACK traffic; vehicular-specific.
Naim et al. (2016) [6]	Heartbeat polling, replicated backup activation	Reactive	Detection-driven; single backup cannot survive correlated failure; heartbeat overhead.
Reyana et al. (2023) [7]	Hybrid PSO-ABC placement optimisation	Preventive (optimisation)	Computationally heavy; reliability is a static fitness term; no per-node failure probability.
Gupta & Singh (2025) [8]	DCLB selection, threshold-triggered proactive migration	Proactive (heuristic)	Threshold hard to set; standby reservation wastes capacity; prediction carries no confidence.
This Work	Causal BBN posterior inference, expected-loss migration, risk-tiered replication	Proactive (probabilistic)	Needs initial CPT parameterisation (seeded from literature priors); validation on a real fog failure trace pending.

E. Inferences Drawn from the Survey

Read together rather than individually, the five systems support six inferences that shape the design proposed in this paper.

1) Fault tolerance in fog computing is overwhelmingly reactive. Three of the five systems act only once a fault has been detected—by a missing heartbeat [6], an unacknowledged message [5], or a threshold breach during execution [4]—so the recovery latency is incurred at precisely the moment the application can least absorb it.
2) Acting before failure demonstrably helps. FTAPA [8] reports measurably reduced service downtime from migrating a microservice ahead of a predicted failure. The principle is sound, and this paper builds on it rather than disputing it.
3) Where prediction is attempted it is deterministic and single-metric. Both Rajab and Younis [4] and Gupta and Singh [8] reduce the decision to a comparison of one scalar against a preset threshold, and the authors of both report the difficulty of choosing that threshold as an open limitation.
4) Node health is treated as directly observable. No surveyed system represents it as a latent condition that must be inferred from indicators which are noisy, mutually dependent, and sometimes absent—despite the fact that a telemetry channel is most likely to fall silent exactly when the node hosting it is degrading.
5) Redundancy is provisioned uniformly rather than by risk. A backup is reserved per task [4], per server [6], or per microservice [8], consuming standby capacity irrespective of whether a failure is likely; Gupta and Singh [8] identify this as a standing cost of their own design.
6) Reliability enters optimisation only as a static term. QoS-EO [7] scores reliability within a fitness function but never estimates the probability that a particular node will fail within a particular interval, so its reliability term cannot respond to changing conditions.

Taken together, these inferences point to a single conclusion: what the field lacks is a probabilistic and explainable estimate of imminent node failure, together with a decision rule that spends migration and redundancy in proportion to that estimate.

F. Selection of the Modelling Approach

Several established formalisms could in principle supply such an estimate, and the choice among them was made against four requirements imposed by the problem: the method must function without a large corpus of labelled failures, since none exists for fog topologies; it must continue to produce an estimate when part of the telemetry is unavailable; it must emit a calibrated probability, because the decision rule proposed in Section III multiplies that probability by a cost; and it must be interpretable, so that an operator can act on the reason for a prediction rather than only its magnitude. Table II summarises the candidates against these requirements.

Fixed threshold rules fail the first three requirements outright. Metaheuristic placement optimises an assignment but never estimates failure at all. Markov chains and fault trees, the classical instruments of reliability analysis, suffer state-space explosion on systems of this size and accommodate partial evidence poorly—precisely the comparison Weber et al. [11] draw in concluding that Bayesian networks have displaced them across dependability and maintenance applications. Discriminative machine learning, including the deep-learning approach that Pang and Abramov [13]

TABLE II: Selection of the Modelling Approach

Candidate Methodology	Requirement Match	Assessment
Fixed Threshold Rules	Inadequate	Fail the first three requirements outright: no uncertainty, no partial-evidence reasoning, no calibrated output.
Metaheuristic Placement (PSO/ABC)	Inadequate	Optimises an assignment but never estimates the probability of failure at all.
Markov Chains / Fault Trees	Partially Adequate	State-space explosion at this system size; accommodate partial evidence poorly.
Discriminative Deep Learning	Partially Adequate	Needs abundant labelled data; must impute absent channels; confidence scores poorly calibrated.
Bayesian Belief Network	Fully Adequate	CPTs seeded from priors and refined online; marginalises over unobserved variables; formal calibrated posterior; causal and interpretable.

apply to IoT reliability, achieves high accuracy where abundant labelled data exists, but requires an input value in every feature slot and so must impute the very channels whose absence is diagnostic; its confidence scores are also well known to be poorly calibrated, which matters when the output is to be multiplied by a cost.

A Bayesian Belief Network satisfies all four requirements simultaneously. Its conditional probability tables can be initialised from expert judgement and published reliability priors and then refined from observation, so it is usable from the first epoch. Inference marginalises over unobserved variables, so partial telemetry degrades the sharpness of the posterior without preventing a decision. The output is a probability in the formal sense, suitable for composition with a loss function. And because the graph encodes causal structure, the same inference that produces the posterior also identifies the parent configuration most responsible for it. This combination, rather than any single property, is why the Bayesian Belief Network was selected.

G. Research Gap

Table I makes the gap explicit. Every surveyed fault-tolerance mechanism either reacts to a fault that has already occurred, or anticipates it using a deterministic comparison against a fixed threshold on a small number of directly observed metrics. None represents node health as a latent variable, none quantifies the uncertainty of its own prediction, none continues to reason when telemetry is incomplete, and none uses a predicted failure probability to modulate how much redundancy a given module deserves. Conversely, the BBN literature provides a mature apparatus for probabilistic reliability inference under uncertainty, but has not been applied to the fog placement problem. This paper addresses that intersection.

III. Proposed Methodology

A. System Architecture

Fig. 1 presents the proposed framework. It preserves the conventional three-tier fog topology—an IoT and end-device layer, a fog layer, and a cloud layer—and introduces a Proactive Fault-Tolerance Controller that is co-located with the Fog Orchestration Node at the proxy tier of the hierarchy, where it has visibility of an entire cluster of fog nodes without incurring the round-trip delay of a cloud-hosted controller.

At the lowest layer, sensors and end devices—electrocardiogram and electroencephalogram sensors, surveillance cameras, and mobile user equipment—emit tuples that are consumed by application modules placed on fog nodes. Every fog node hosts a lightweight telemetry agent that periodically reports the observable indicators of node condition. The controller consumes this telemetry through the four stages drawn inside it in Fig. 1. A pre-processing and discretisation stage cleans, aligns, and bins the continuous measurements into the ordinal states over which the network is defined. The BBN inference engine (N1) then computes, for each node, the posterior probability of failure within the look-ahead horizon. The risk-aware placement and migration module (N2) converts those posteriors into placement and migration decisions, and the criticality-aware replication module (N3) decides how much standby capacity each placed module should be granted. Decisions are actuated back onto the fog nodes as module placements, live migrations, and replica allocations, while modules that no fog node can safely host are offloaded to the cloud. Observed outcomes—whether a node predicted to fail actually failed—are fed back to update the conditional probability tables, and the updated tables are synchronised with a cloud-hosted repository so that knowledge learned in one cluster benefits others.

B. Novelty 1: BBN-Based Failure Prediction Engine

The core of the framework is a Bayesian Belief Network over fog-node condition, whose structure is shown in Fig. 2. The network is organised in three tiers. Observable evidence variables correspond directly to telemetry: CPU utilisation, RAM utilisation, tuple queue depth, uplink latency, jitter and packet loss, residual energy or battery level,

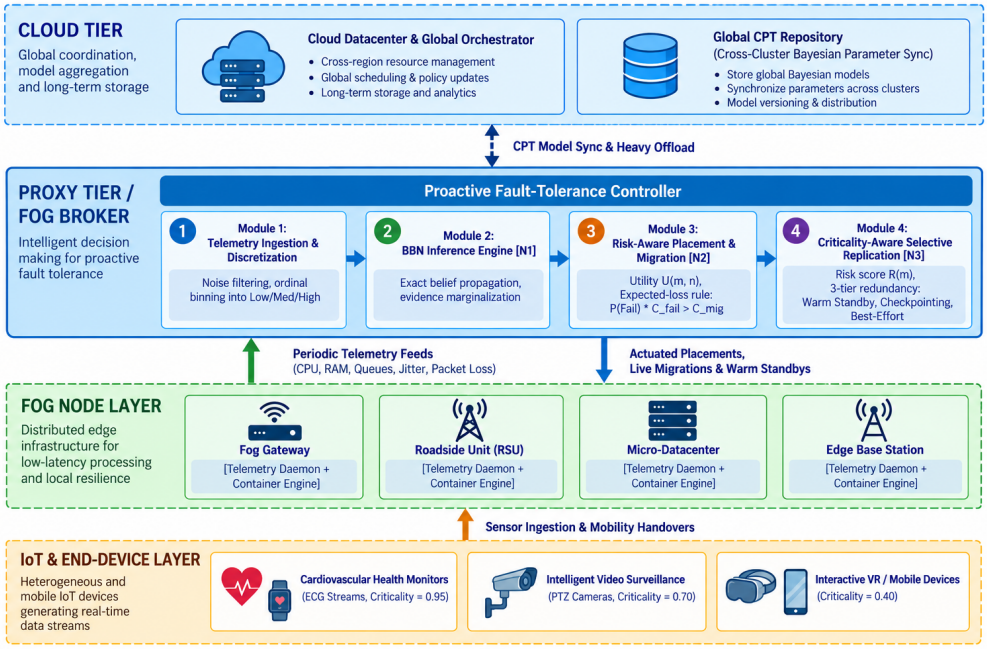


Fig. 1: Block diagram of the proposed BBN-driven proactive fault-tolerance framework. Telemetry from the fog layer is discretised and supplied as evidence to the BBN inference engine (N1); the resulting posterior drives risk-aware placement and expected-loss migration (N2) and criticality-aware selective replication (N3). Observed outcomes are fed back for online conditional-probability-table adaptation.

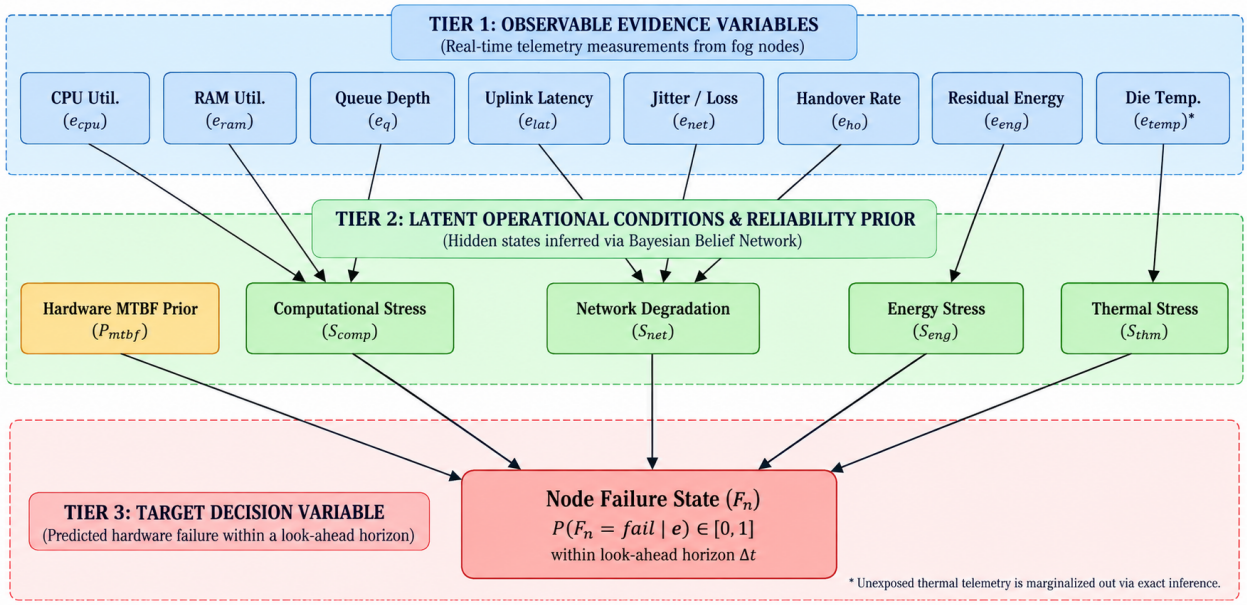


Fig. 2: Structure of the Bayesian Belief Network over fog-node condition. Observable telemetry forms the evidence tier; four latent variables aggregate it into interpretable conditions; the target is failure of the node within the horizon Δt, additionally conditioned on a node-reliability prior derived from the device class and its historical mean time between failures. Evidence that is unavailable at a decision epoch is marginalised out rather than imputed.

ambient or die temperature, and the handover rate induced by user mobility. Intermediate latent variables aggregate these into interpretable conditions—computational stress, network degradation, energy stress, and thermal stress. A single target variable, the failure of the node within the horizon, is the child of the intermediate tier, additionally conditioned on a node-reliability prior derived from the device class and its historical mean time between failures.

The edges follow the physical origin of each indicator. CPU utilisation, RAM utilisation, and tuple queue depth are

the parents of computational stress; uplink latency, jitter with packet loss, and handover rate are the parents of network degradation; residual energy is the parent of energy stress; and temperature is the parent of thermal stress. Those four latent variables, together with the node-reliability prior, are in turn the parents of the target. Each variable is discretised into a small ordinal state space, $\{\text{low}, \text{medium}, \text{high}\}$, so that conditional probability tables remain compact and interpretable. Given the set of evidence actually observed at a decision epoch, the engine computes the posterior

$$P(F = \text{fail} \mid e) = \frac{\sum_u \prod_{V \in \mathcal{V}} P(V \mid \text{pa}(V))}{P(e)} \tag{1}$$

where $F \in \{\text{fail}, \text{not-fail}\}$ is the target variable and fail means that the node becomes unable to host its modules at some point within the next Δt; e is the set of observed states of those telemetry variables that actually reported at this decision epoch (channels that failed to report are simply absent from e rather than imputed); $\mathcal{V}$ denotes all fourteen variables of the network—the eight evidence variables, the four latent condition variables, the node-reliability prior, and F itself—so that the product $\prod_V P(V \mid \text{pa}(V))$ reconstitutes the full joint distribution from the local conditional probability tables; u ranges over every joint assignment of states to the unobserved variables, namely the latent conditions plus any telemetry channel missing from e; and $P(e)$ is the normalising constant obtained by summing the same expression over both states of F as well as over u, so that the two posterior probabilities sum to one.

The summation over u is the step that gives the network its tolerance of missing telemetry: every possible combination of values for the variables that were not observed is enumerated and its contribution added up, which is what marginalising them out means. This is the property that distinguishes the approach from a threshold rule and from a discriminative classifier: a missing telemetry channel widens the sum over u and so degrades the sharpness of the posterior, but it does not prevent a decision from being made, whereas a threshold on an unreported metric simply never fires.

With one exception, every evidence variable is derived from a quantity the simulator described in Section IV already maintains. CPU utilisation is allocated MIPS against host capacity per scheduling interval; RAM utilisation comes from the memory provisioner; queue depth is the pending tuple queue on the device; uplink latency and jitter combine the configured link delay with queueing delay; residual energy is accumulated consumption against a configured budget; and handover rate is the count of parent reassignments per window, while the node-reliability prior is fixed by device class and configured mean time between failures. Temperature is read wherever a node exposes a thermal sensor; where none is exposed, as on the simulated devices of Section IV, that variable is left unobserved and is marginalised out by (1)—the partial-evidence case the network is built to absorb, rather than a gap that has to be filled in. Continuous quantities are discretised using quantiles of the baseline runs, so that a state such as high CPU utilisation is defined relative to the observed distribution of this topology rather than by an arbitrary cut-off.

Conditional probability tables are initialised from the reliability literature [10], [11], [13] and from expert elicitation over the device classes present in the topology, since no public failure trace exists for the fog topology under study. They are subsequently refined online: each prediction is paired with the outcome eventually observed, and the relevant table entries are updated by Bayesian parameter learning with Dirichlet priors, so the model becomes progressively better calibrated to the deployment it governs. Because the network is causal rather than correlational, the posterior is accompanied by the most influential parent configuration, giving an explanation of the form “failure probability is high principally because network degradation is high given elevated jitter and packet loss”—information that supports both operator trust and the choice of remedy.

C. Novelty 2: Risk-Aware Placement and Expected-Loss Migration

The posterior produced by the BBN is not consumed as a binary alarm. For placement, the controller scores each candidate node for a given module by a normalised multi-objective utility

$$U(m, n) = \alpha \cdot \Big(1 - \hat{L}(m, n)\Big) + \beta \cdot \Big(1 - \hat{E}(m, n)\Big) + \gamma \cdot (1 - P(F_n = \text{fail} \mid e)) \tag{2}$$

where $U(m, n) \in [0, 1]$ is the placement utility of hosting module m on node n; $\hat{L}(m, n)$ and $\hat{E}(m, n)$ are the predicted end-to-end latency and energy cost of running m on n, normalised across the candidate nodes so that low latency and low energy raise the utility; $1 - P(F_n = \text{fail} \mid e)$ is the reliability of node n as the network currently estimates it; and α, β, γ are non-negative weights on latency, energy, and risk, summing to 1 and configured per application class. The controller places each module on the feasible node of highest utility. The weights are what make the utility application-aware: a latency-critical health module is configured with a large γ, so it is steered away from nodes the network considers fragile even when they are computationally attractive, whereas a best-effort module weights latency and energy more heavily instead.

For migration, the framework replaces the fixed fault threshold of prior work with a cost-sensitive decision. Migrating a running module is not free: it consumes bandwidth, incurs a brief service interruption, and may place the module

on a less favourable node. Migration is therefore triggered only when the expected loss from remaining exceeds the cost of moving,

$$P(F_n = \text{fail} \mid e) \cdot C_{\text{fail}}(m) > C_{\text{mig}}(m, n \to n') \tag{3}$$

where the left-hand side is the expected loss of leaving m on its current node n for one more decision epoch, being the probability of the loss multiplied by its magnitude; $C_{\text{fail}}(m)$ is the penalty attributable to losing module m—itself a function of the application's criticality and deadline, so that a cardiac classifier carries a far larger value than a best-effort module; $C_{\text{mig}}(m, n \to n')$ is the estimated cost of relocating m to the best alternative node n', covering the bandwidth needed to transfer state, the brief interruption to service, and any utility given up by moving to a less favourable node, where n' maximises $U(m, n)$ of (2) among the feasible candidates.

The decision rule is self-calibrating: a module whose failure is cheap tolerates a high failure probability before it is moved, while a module whose failure is expensive is moved on comparatively weak evidence. Rearranging (3) makes this explicit—the module migrates once $P(F_n = \text{fail} \mid e)$ exceeds the ratio $C_{\text{mig}}/C_{\text{fail}}$—so the effective threshold is derived per module from the relative cost of failing and moving. This directly answers the threshold-selection difficulty acknowledged by Rajab and Younis [4] and Gupta and Singh [8], because there is no longer a single global threshold to select.

D. Novelty 3: Criticality-Aware Selective Replication

Prior schemes provision a backup for every protected task [4], [6] or pre-select a backup node for every placed microservice [8]. The standby capacity so reserved is consumed whether or not a failure occurs, which Gupta and Singh [8] identify as a standing cost of their design. The proposed framework instead treats redundancy as a resource to be allocated where it changes the outcome. Each module is assigned a criticality weight reflecting the consequence of its loss, and the controller computes a risk score

$$R(m) = P(F_n = \text{fail} \mid e) \cdot \kappa(m) \tag{4}$$

where $R(m)$ is the risk score of module m—the expected consequence of its loss over the coming horizon, and the quantity on which its redundancy tier is decided; $P(F_n = \text{fail} \mid e)$ is the posterior failure probability from (1) for the node n that currently hosts m; and $\kappa(m) \in [0, 1]$ is the criticality weight of m, reflecting the consequence of losing it—near 1 for a cardiac arrhythmia classifier, near 0 for a best-effort module. It is the same criticality that enters $C_{\text{fail}}(m)$ in (3).

Modules are then assigned to one of three redundancy tiers by comparing $R(m)$ against two boundaries τ_{high} and τ_{mid}.

- Warm standby: High-risk, high-criticality modules, for which $R(m) \geq \tau_{\text{high}}$—for example a cardiac arrhythmia classifier on a node the network judges fragile—receive a warm standby on a pre-selected low-risk node, with state synchronised so that failover is immediate.
- Periodic checkpointing: Modules of intermediate risk, $\tau_{\text{mid}} \leq R(m) < \tau_{\text{high}}$, receive periodic state checkpointing without a dedicated standby, so that recovery requires re-instantiation but not re-execution.
- Best effort: Low-risk modules, for which $R(m) < \tau_{\text{mid}}$, receive no redundancy and are simply restarted if they fail.

Because tier assignment is recomputed as posteriors change, standby capacity follows risk dynamically rather than being pinned at placement time, and the aggregate redundancy overhead is bounded by a configurable capacity budget B that the controller respects when admitting modules to the highest tier: where the budget is exhausted, the modules of greatest $R(m)$ hold the warm standbys.

E. End-to-End Workflow

The framework operates as a continuous control loop over discrete decision epochs, summarised in Fig. 3.

1) Telemetry agents on every fog node report the observable indicators to the controller.
2) The pre-processing stage aligns, cleans, and discretises the measurements into the ordinal states of the evidence variables, flagging any channel that failed to report.
3) The BBN inference engine computes the posterior failure probability for every node by (1), marginalising over unobserved evidence, and records the dominant causal explanation.
4) For newly arriving modules, the risk-aware placement policy ranks candidate nodes by the multi-objective utility (2) and places each module on the highest-scoring feasible node.
5) For running modules, the expected-loss rule (3) is evaluated; modules for which the expected failure loss exceeds the migration cost are migrated pre-emptively to the best alternative node.
6) The replication manager recomputes the risk scores (4), assigns modules to redundancy tiers, and provisions or releases standby capacity subject to the redundancy budget.

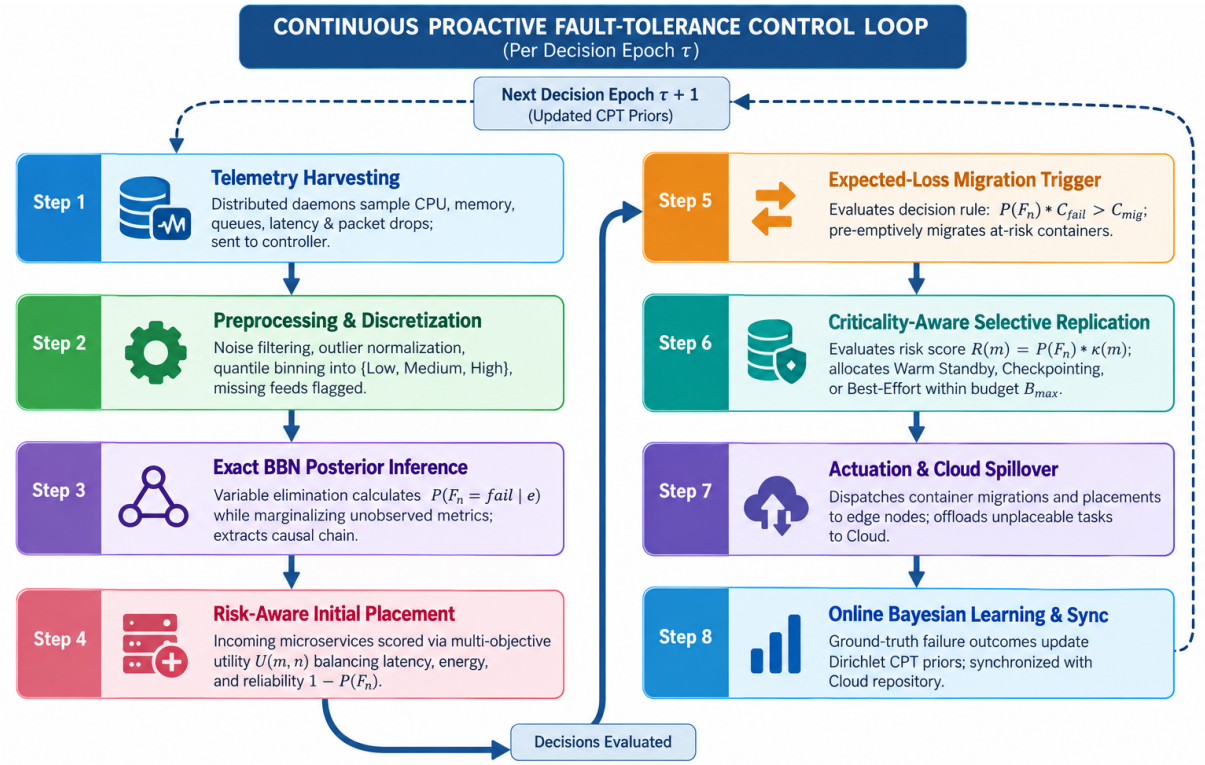


Fig. 3: End-to-end control loop of the proposed framework over one decision epoch. Steps 1–3 perform inference, steps 4–6 convert the posterior into placement, migration, and replication decisions, step 7 actuates those decisions on the fog layer and offloads whatever the fog cannot host, and step 8 closes the loop by updating the conditional probability tables from observed outcomes and synchronising them with the cloud repository, so that the model becomes progressively better calibrated to the deployment it governs.

7) Actuated decisions are applied to the fog layer; modules that no fog node can safely host are offloaded to the cloud.
8) Observed outcomes are compared against predictions, the conditional probability tables are updated by Bayesian parameter learning, and the revised tables are synchronised with the cloud repository before the next epoch begins.

IV. Experimental Setup and Dataset Description

A. Simulation Environment

The framework is being implemented as an extension to iFogSim2 [1], the widely used discrete-event simulator for edge and fog resource management, which builds on CloudSim and on the original iFogSim toolkit [2]. iFogSim2 is selected because it already provides the three capabilities the evaluation requires: mobility support driven by real and synthetic location traces, microservice orchestration with horizontal scaling and service discovery, and dynamic clustering of fog nodes. The proposed controller is realised as a new placement logic registered with the microservices controller, so that it can be compared against the simulator's existing placement policies under identical topologies and workloads. Because iFogSim2 does not model node failure natively, a fault-injection harness is being added that fails nodes according to configurable stochastic processes conditioned on their simulated load, energy, and link state, thereby generating both the failure events to be predicted and the ground truth against which predictions are scored.

B. Dataset Description

Three categories of data drive the evaluation. The physical topology is taken from the Melbourne central business district edge-resource dataset distributed with iFogSim2, which enumerates 131 edge and fog resources with their geographic coordinates and parent-child relationships across a four-level hierarchy of cloud data centre, proxy server, gateway, and end user. User mobility is supplied both by a real trace of 87 recorded positions and by five synthetic traces of 100 positions each, generated by the simulator's random mobility generator; mobility matters here because handover

is one of the evidence variables in the network and a driver of load redistribution. Application workloads comprise three models: a cardiovascular health-monitoring application structured as a client module feeding two downstream analysis microservices, an intelligent surveillance application in which a motion detector, object detector, and object tracker drive a pan-tilt-zoom actuator, and a virtual-reality gaming application in which electroencephalogram data is classified and fed to a global coordinator. A configuration file accompanying the microservice application supplies ten distinct application profiles specifying per-microservice processing requirements, inter-module network loads, per-hop latencies, throughput, and budget constraints, and these parameters define the operating envelope from which the evidence variables are derived.

It should be stated plainly that no public failure-trace dataset exists for a fog topology of this kind. Failure events are therefore synthesised by the injection harness described above, and the conditional probability tables are seeded from priors reported in the reliability literature [10], [11], [13] before being adapted online. Validating the framework against a real fog failure trace, should one become available, is identified as future work.

C. Evaluation Metrics and Baselines

The framework will be assessed along two axes. Prediction quality will be measured by precision, recall, and F1 score of failure prediction within the look-ahead horizon, by the calibration of the posterior against observed failure frequency, and by lead time—the interval between a prediction crossing the decision rule and the actual failure—since a prediction that arrives too late to act upon has no operational value. System quality will be measured by mean and tail end-to-end application loop delay, deadline-violation ratio for critical loops, total energy consumption across cloud, fog, and end-device tiers, network usage, execution cost, service downtime per injected failure, and redundancy overhead expressed as reserved standby capacity. Robustness will be probed by withholding telemetry channels at inference time to quantify the degradation of prediction quality under partial evidence.

Four baselines will be compared under identical conditions: cloud-only placement; the default edgeward placement of iFogSim2 with no fault tolerance; reactive checkpointing with primary-backup replication in the manner of Rajab and Younis [4]; and threshold-based proactive migration in the manner of FTAPA [8], which is the closest competitor and the most informative comparison, since it isolates the contribution of probabilistic inference from the contribution of acting early.

V. Conclusion and Future Work

This paper has argued that fault tolerance in fog computing remains predominantly reactive, and that the proactive mechanisms so far reported reduce an inherently uncertain judgement to a fixed threshold on a single metric. A survey of five representative fog fault-tolerance systems found that none quantifies the probability of imminent node failure, none reasons under partial telemetry, and none modulates redundancy by predicted risk. Drawing on an established body of work applying Bayesian Belief Networks to reliability and dependability, the paper has proposed a framework in which a causal BBN infers the posterior probability of fog node failure from noisy and possibly incomplete evidence, a risk-aware utility and an expected-loss decision rule convert that posterior into placement and pre-emptive migration decisions, and a criticality-aware scheme allocates standby capacity in proportion to risk and consequence. The architecture, its block diagram, and its workflow have been presented, together with the simulation environment and the datasets on which it will be evaluated.

Work in Phase II will proceed in four stages: construction and expert validation of the BBN structure and its initial conditional probability tables; implementation of the inference engine and the fault-injection harness within iFogSim2; implementation of the risk-aware placement, expected-loss migration, and selective replication policies as an iFogSim2 placement logic; and comparative evaluation against the four baselines across the metrics enumerated in Section IV. Longer-term extensions include a dynamic Bayesian network formulation to model the temporal evolution of node degradation explicitly, and distributed inference across clustered fog nodes to remove the controller as a single point of failure.

References

[1] R. Mahmud, S. Pallewatta, M. Goudarzi, and R. Buyya, “iFogSim2: An extended iFogSim simulator for mobility, clustering, and microservice management in edge and fog computing environments,” Journal of Systems and Software, vol. 190, p. 111351, 2022.

[2] H. Gupta, A. V. Dastjerdi, S. K. Ghosh, and R. Buyya, “iFogSim: A toolkit for modeling and simulation of resource management techniques in the Internet of Things, Edge and Fog computing environments,” Software: Practice and Experience, vol. 47, no. 9, pp. 1275–1296, 2017.

[3] D. A. Hasan and S. R. M. Zeebaree, “Proactive fault tolerance in distributed cloud systems: A review of predictive and preventive techniques,” Indonesian Journal of Computer Science, vol. 13, no. 2, 2024.

[4] H. T. Rajab and M. F. Younis, “Dynamic fault tolerance aware scheduling for healthcare system on fog computing,” Iraqi Journal of Science, pp. 308–318, 2021.

[5] S. A. Syed, M. Rashid, S. Hussain, F. Azim, H. Zahid, A. Umer, A. Waheed, M. Zareei, and C. Vargas-Rosales, “QoS aware and fault tolerance based software-defined vehicular networks using cloud-fog computing,” Sensors, vol. 22, no. 1, p. 401, 2022.

[6] M. H. Naim, J. M. Zain, and K. A. Jalil, "Fault tolerance mechanism for software application through fog computing as middleware," International Journal of Computing and Digital Systems, vol. 5, no. 3, 2016.
[7] A. Reyana, S. Kautish, K. A. Alnowibet, H. M. Zawbaa, and A. W. Mohamed, "Opportunities of IoT in fog computing for high fault tolerance and sustainable energy optimization," Sustainability, vol. 15, no. 11, p. 8702, 2023.
[8] S. Gupta and N. Singh, "Optimized fault-tolerant placement of microservices in distributed fog networks," International Journal of Data Informatics and Intelligent Computing, vol. 4, no. 4, pp. 52–69, 2025.
[9] J. Pearl, Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference. San Mateo, CA: Morgan Kaufmann, 1988.
[10] N. Fenton, M. Neil, and D. Marquez, "Using Bayesian networks to predict software defects and reliability," in Proc. 5th Int. Conf. Mathematical Methods in Reliability (MMR07), Glasgow, UK, 2007.
[11] P. Weber, G. Medina-Oliva, C. Simon, and B. Iung, "Overview on Bayesian networks applications for dependability, risk analysis and maintenance areas," Engineering Applications of Artificial Intelligence, vol. 25, no. 4, pp. 671–682, 2012.
[12] P. A. Aguilera, A. Fernández, R. Fernández, R. Rumí, and A. Salmerón, "Bayesian networks in environmental modelling," Environmental Modelling & Software, vol. 26, no. 12, pp. 1376–1388, 2011.
[13] B. Pang and E. S. Abramov, "Reliability analysis and parameter selection for IoT communication based on deep learning," Eng, vol. 6, no. 8, p. 171, 2025.